Predictive Maintenance from First Principles: Weibull MLE with Co-variates

What Survival Analysis Knows That Your Regression Model Doesn't-

A Step-by-Step Derivation of the Likelihood, Gradients, and Hessian Without Touching Autograd

Imagine that you are monitoring a fleet of 10,000 industrial tools. After six months, 339 have broken down. You want to know: *which operating conditions cause failure and by how much?* A standard regression model on those 339 observations would answer a different question: one about the most fragile machines, not the fleet. The 9,661 tools that are still running at the end of the study also carry information. Throwing them out is statistically indefensible; treating them as failures is worse.

This is the censoring problem, and it is the reason an entire branch of statistics, survival analysis, exists. The workhorse of industrial survival analysis is the **Weibull distribution**. In this article, we build the Weibull regression model from scratch: we explain why it is the right tool, derive the MLE objective and every gradient and Hessian entry analytically, and implement Newton-Raphson without touching autograd.

1 Why Standard ML Fails: The Censoring Problem

In a predictive maintenance setting, sensors stream real-time telemetry, for example, air temperature, process temperature, rotational speed, and torque for every machine in a fleet. The goal is to model *time to failure* as a function of these covariates. Why can't we just run a regression of failure time on sensor readings?

Because most machines haven't failed yet. When the study window closes, a machine that is still operational has a *partial* observation: we know it survived at least until time t_i, but we do not know its true failure time. In survival analysis, this is called **right-censoring**.

Standard regression forces an unacceptable choice: discard censored observations (selection bias, studying only the most fragile machines) or treat them as failures at the study endpoint (severe underestimation of lifetime). Both options corrupt the parameter estimates. Survival analysis resolves this by writing a *likelihood that conditions on what was actually observed, an exact failure for some machines, a lower bound for others and maximizing that likelihood jointly.*

2 Why Weibull? Physics Encoded in a Distribution

Before we write down any formulas, the honest question is: why *this* parametric family and not, say, log-normal or exponential?

The answer is the **hazard function**. Define $h(t)$ as the instantaneous failure rate given survival to time t:

$$h(t) = \lim_{\Delta t \to 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}$$

Real machinery follows one of three failure regimes, and each leaves a distinct hazard signature:

- **Infant-mortality failures** cluster early in life (manufacturing defects, improper installation). Hazard is high at $t = 0$ and *decreasing*: $h'(t) < 0$.
- **Random failures** occur at a constant rate, independent of age. Hazard is flat: $h'(t) = 0$. This is the memoryless exponential model.

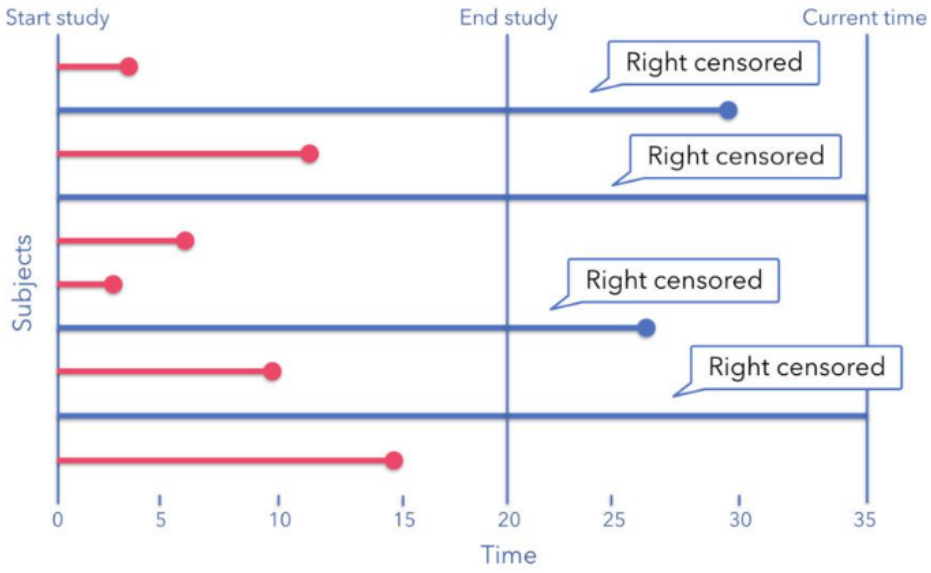


Figure 1: Right-censoring illustrated. Machines ending in $\times$ contribute their exact failure time; those ending in $\circ$ contribute only the information that they survived to at least t_i.(figure not drawn to scale)

- **Wear-out failures** accelerate with cumulative stress. Hazard *increases*: $h'(t) > 0$.

The Weibull distribution is the unique two-parameter family that can represent all three regimes by varying a single shape parameter β. Its hazard function is:

$$h(t \mid \alpha, \beta) = \frac{\beta}{\alpha}\left(\frac{t}{\alpha}\right)^{\beta-1}$$

When $\beta < 1$, $h(t)$ decreases (infant mortality). When $\beta = 1$, $h(t)$ is constant (exponential special case). When $\beta > 1$, $h(t)$ increases (wear-out).

This is not curve-fitting convenience. In manufacturing, β is a *physics quantity*: a tool operating in the wear-out regime has $\beta > 1$ by design, and estimating exactly how large β is tells engineers how fast the failure rate is accelerating.

3 The Weibull Regression Model

Rather than fitting a single Weibull distribution to the entire fleet, we let the shape parameter vary per machine as a function of its sensor environment. For observation i:

$$t_i \sim \text{Weibull}(\alpha,\, \beta_i)$$

The **scale parameter** $\alpha > 0$ sets the characteristic lifetime: 63.2% of components fail before time α (when $\beta = 1$). Think of it as the "fleet baseline." The **shape parameter** $\beta_i > 0$ is machine-specific and encodes how sensor conditions accelerate or retard aging.

To link covariates to β_i, we need a function that maps $\mathbb{R}^p \to (0, \infty)$. A linear function is inadmissible

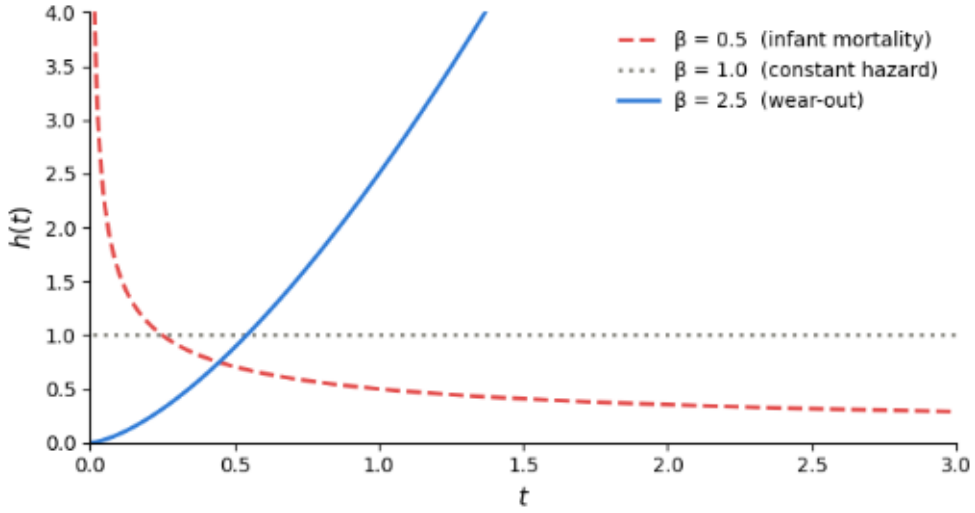


Figure 2: The Weibull hazard function for three values of β. The shape parameter encodes the physical aging regime of the component.

, it can go negative. The natural choice is the exponential link:

$$\beta_i = \exp\left(\sum_{j=1}^{p} \gamma_j x_{ij}\right) = \exp(\boldsymbol{\gamma}^\top \boldsymbol{x}_i)$$

where $\boldsymbol{x}_i \in \mathbb{R}^p$ are the sensor readings for machine i (no intercept column, for reasons we prove in Section 10) and $\boldsymbol{\gamma} \in \mathbb{R}^p$ is the coefficient vector to be learned.

Each γ_j has a clean interpretation: a unit increase in sensor j multiplies β_i by e^{γ_j}, stretching or compressing the failure curve on a log scale.

The full parameter set is $\boldsymbol{\theta} = [\alpha, \boldsymbol{\gamma}^\top]^\top \in \mathbb{R}^{1+p}$.

4 Why MLE? The Right Framework for Censored Data

We want to estimate $\boldsymbol{\theta}$ from data. Why Maximum Likelihood Estimation and not, say, method of moments or Bayesian inference?

Two reasons. First, MLE naturally handles the mixed structure of censored data. Each observation contributes a *different type* of information: a failed machine contributes a point likelihood $f(t_i)$ while a censored machine contributes a tail probability $S(t_i)$. MLE accommodates both through a unified likelihood that weights each observation by exactly what was observed.

Second, the Weibull NLL is **strictly convex** over the feasible parameter space when the design matrix has full column rank (proved in Section 9). This means the MLE is unique, always exists, and any convergent optimiser will find it. We are not guessing at a local minimum there is only one minimum.

5 Deriving the Negative Log-Likelihood, Step by Step

Step 1: Core Weibull functions. The probability density function (PDF) gives the instantaneous likelihood of failing at exactly time t_i:

$$f(t_i \mid \alpha, \beta_i) = \frac{\beta_i}{\alpha} \left(\frac{t_i}{\alpha}\right)^{\beta_i - 1} \exp\left(-\left(\frac{t_i}{\alpha}\right)^{\beta_i}\right)$$

The survival function gives the probability of *still running* past time t_i:

$$S(t_i \mid \alpha, \beta_i) = \exp\left(-\left(\frac{t_i}{\alpha}\right)^{\beta_i}\right)$$

Note that $S(t_i) = 1 - F(t_i)$, i.e. it is the complementary CDF.

Step 2: Building the joint likelihood under censoring. Introduce a binary indicator $\delta_i \in \{0, 1\}$: it equals 1 if machine i suffered a confirmed failure, and 0 if the observation is right-censored. A failed machine contributes the density at its moment of death; a censored machine contributes only the probability it survived to at least t_i:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} [f(t_i)]^{\delta_i} [S(t_i)]^{1-\delta_i}$$

Step 3: Substitute and collect the exponential. Plugging in the explicit expressions and collecting the shared factor $e^{-(t_i/\alpha)^{\beta_i}}$ from both terms:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} \left[\frac{\beta_i}{\alpha} \left(\frac{t_i}{\alpha}\right)^{\beta_i - 1}\right]^{\delta_i} \exp\left(-\left(\frac{t_i}{\alpha}\right)^{\beta_i}\right)$$

Step 4: Take the log. Products become sums:

$$\ln L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \delta_i \left[\ln \beta_i - \ln \alpha + (\beta_i - 1) \ln\left(\frac{t_i}{\alpha}\right)\right] - \sum_{i=1}^{n} \left(\frac{t_i}{\alpha}\right)^{\beta_i}$$

Step 5: Flip the sign. Standard optimisers minimise. Multiplying by -1 gives the **Negative Log-Likelihood (NLL)**:

$$\boxed{\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \left[\delta_i \left(\ln \beta_i - \ln \alpha + (\beta_i - 1) \ln\left(\frac{t_i}{\alpha}\right)\right) - \left(\frac{t_i}{\alpha}\right)^{\beta_i}\right]}$$

This is the surface we navigate to find $\hat{\boldsymbol{\theta}}$.

6 Gradient Descent vs. Newton-Raphson on a Survival Surface

To keep the algebra transparent, we narrow to a **single-sensor model**: $\beta_i = \exp(\gamma_0 + \gamma_1 x_i)$, giving three parameters $\boldsymbol{\theta} = [\alpha, \gamma_0, \gamma_1]^\top$.

Gradient Descent. Gradient Descent reads only the local slope and steps downhill at a fixed rate η:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \, \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

The weakness is geometric blindness. A small gradient might signal a flat plateau (genuine lack of information) or it might signal that the step is oscillating across a narrow ravine. Survival likelihood surfaces typically have both features: steep ravines near the optimum, broad flat regions far from it. Without curvature information, GD bounces or crawls depending on η.

Newton-Raphson. Newton-Raphson adds **curvature information** via the Hessian $H = \nabla^2 \mathcal{L}$:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - H^{-1}(\boldsymbol{\theta}_k) \, \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Where does this update come from? Expand $\mathcal{L}$ in a second-order Taylor series around $\boldsymbol{\theta}_k$:

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \tfrac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^\top H (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$$

Setting the gradient of this quadratic approximation to zero and solving for $\boldsymbol{\theta}$ gives exactly the Newton-Raphson update. In other words, each Newton step minimises the local quadratic approximation of the surface exactly. On the Weibull NLL, which is smooth and strictly convex, this approximation is nearly exact near the optimum, producing convergence in fewer than ten iterations versus thousands for GD.

7 Analytical Derivations: Gradient and Hessian

Throughout, we define recurring building blocks:

$$\tau_i \;=\; \ln\!\left(\frac{t_i}{\alpha}\right), \qquad u_i \;=\; \left(\frac{t_i}{\alpha}\right)^{\beta_i}$$

7.1 First-Order Derivatives (Gradient Vector)

With respect to α. Differentiating $\mathcal{L}$ and applying the chain rule to $(t_i/\alpha)^{\beta_i}$:

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \sum_{i=1}^{n} \frac{\beta_i}{\alpha} [\delta_i - u_i]$$

Intuition: Failed machines ($\delta_i = 1$) pull α upward toward longer predicted lifetimes. Machines whose cumulative hazard u_i exceeds the failure indicator push it down.

With respect to γ_0. Because $\beta_i = e^{\gamma_0 + \gamma_1 x_i}$, the chain rule gives $\partial \beta_i / \partial \gamma_0 = \beta_i$:

$$\frac{\partial \mathcal{L}}{\partial \gamma_0} = -\sum_{i=1}^{n} [\delta_i (1 + \beta_i \tau_i) - \beta_i \, \tau_i \, u_i]$$

With respect to γ_1. Here $\partial \beta_i / \partial \gamma_1 = \beta_i x_i$, introducing an extra factor of x_i:

$$\frac{\partial \mathcal{L}}{\partial \gamma_1} = -\sum_{i=1}^{n} x_i [\delta_i (1 + \beta_i \tau_i) - \beta_i \, \tau_i \, u_i]$$

Note the structural symmetry: $\partial \mathcal{L} / \partial \gamma_1 = x_i \cdot \partial \mathcal{L} / \partial \gamma_0$ element-wise. This propagates directly into the Hessian cross-terms.

7.2 Second-Order Derivatives (Hessian Matrix)

Introduce one further shorthand: $v_i = \beta_i \, \tau_i \, u_i$.

$\partial^2 \mathcal{L} / \partial \alpha^2$.

$$\frac{\partial^2 \mathcal{L}}{\partial \alpha^2} = \sum_{i=1}^{n} \frac{\beta_i}{\alpha^2} [-\delta_i + (\beta_i + 1) \, u_i]$$

$\partial^2 \mathcal{L} / \partial \gamma_0^2$.

$$\frac{\partial^2 \mathcal{L}}{\partial \gamma_0^2} = -\sum_{i=1}^{n} [\delta_i \, \beta_i \, \tau_i - v_i (1 + \beta_i \, \tau_i)]$$

$\partial^2 \mathcal{L} / \partial \gamma_1^2$.

$$\frac{\partial^2 \mathcal{L}}{\partial \gamma_1^2} = -\sum_{i=1}^{n} x_i^2 [\delta_i \, \beta_i \, \tau_i - v_i (1 + \beta_i \, \tau_i)]$$

Cross-derivatives.

$$\frac{\partial^2 \mathcal{L}}{\partial \alpha \, \partial \gamma_0} = \sum_{i=1}^{n} \frac{\beta_i \, \tau_i}{\alpha} [\delta_i - (1 + \beta_i \tau_i) \, u_i]$$

The remaining cross-terms follow from the element-wise symmetry:

$$\frac{\partial^2 \mathcal{L}}{\partial \gamma_0 \, \partial \gamma_1} = \sum_{i=1}^{n} x_i \cdot \frac{\partial^2 \mathcal{L}}{\partial \gamma_0^2} \bigg|_i, \qquad \frac{\partial^2 \mathcal{L}}{\partial \alpha \, \partial \gamma_1} = \sum_{i=1}^{n} x_i \cdot \frac{\partial^2 \mathcal{L}}{\partial \alpha \, \partial \gamma_0} \bigg|_i$$

Assembling the six unique entries (symmetric by Schwarz's theorem):

$$H = \begin{bmatrix} \dfrac{\partial^2 \mathcal{L}}{\partial \alpha^2} & \dfrac{\partial^2 \mathcal{L}}{\partial \alpha \, \partial \gamma_0} & \dfrac{\partial^2 \mathcal{L}}{\partial \alpha \, \partial \gamma_1} \\ \cdot & \dfrac{\partial^2 \mathcal{L}}{\partial \gamma_0^2} & \dfrac{\partial^2 \mathcal{L}}{\partial \gamma_0 \, \partial \gamma_1} \\ \cdot & \cdot & \dfrac{\partial^2 \mathcal{L}}{\partial \gamma_1^2} \end{bmatrix}$$

8 Python Implementation: Math to Code, Line by Line

The implementation below is a direct translation of the formulas above. We split it into four blocks to show the correspondence between math and code.

Block 1: Setup and recurring quantities.

```python
import numpy as np

def manual_weibull_nll(X, t, delta, theta):
    """
    X:     (n, p)  sensor matrix     NO intercept column
    t:     (n,)    observed times (failure or censoring)
    delta: (n,)    1 = failure, 0 = censored
    theta: (1+p,)  [alpha, gamma_1, ..., gamma_p]
    """
```

```
    alpha = theta[0]
    gamma = theta[1:]

    # Exponential link: beta_i = exp(X_i . gamma)  [strictly positive]
    beta = np.exp(np.clip(np.dot(X, gamma), -10, 10))

    # Recurring building blocks from the derivation
    tau   = np.log(t / alpha)          # tau_i
    u     = (t / alpha) ** beta        # u_i

    nll = -np.sum(delta * (np.log(beta) - np.log(alpha)
                           + (beta - 1) * tau) - u)
    return nll
```

Block 2: Gradient.

```
def gradient(X, t, delta, theta):
    alpha = theta[0];  gamma = theta[1:]
    beta  = np.exp(np.clip(np.dot(X, gamma), -10, 10))
    tau   = np.log(t / alpha)
    u     = (t / alpha) ** beta

    # d(NLL)/d(alpha)
    d_alpha = np.sum((delta * beta) / alpha
                     - (beta / alpha) * u)

    # Shared factor for gamma derivatives (see derivation)
    common_g = delta * (1.0 + beta * tau) - beta * tau * u
    d_gamma  = -np.dot(X.T, common_g)      # shape (p,)

    return np.concatenate(([d_alpha], d_gamma))
```

Block 3: Hessian.

```
def hessian(X, t, delta, theta):
    alpha = theta[0];  gamma = theta[1:]
    n, p  = X.shape
    beta  = np.exp(np.clip(np.dot(X, gamma), -10, 10))
    tau   = np.log(t / alpha)
    u     = (t / alpha) ** beta

    # d2/d(alpha)2
    d_alpha2  = np.sum(-(delta * beta) / alpha**2
                       + (beta * (beta + 1.0) / alpha**2) * u)

    # Shared factor for gamma2 and cross terms
    common_h  = (delta * beta * tau
                 - beta * tau * u * (1.0 + beta * tau))
    d_gamma2  = -np.dot(X.T * common_h, X)   # (p, p)

    common_cross   = ((delta * beta / alpha) * tau
                      - (beta / alpha) * u * (1.0 + beta * tau))
    d_alpha_gamma  = np.dot(X.T, common_cross)    # (p,)

    H = np.zeros((1 + p, 1 + p))
    H[0,  0]  = d_alpha2
```

```
    H[0, 1:]  = d_alpha_gamma
    H[1:, 0]  = d_alpha_gamma
    H[1:, 1:] = d_gamma2
    return H
```

Block 4: Newton-Raphson loop.

```
def newton_raphson(X, t, delta, max_iter=15, tol=1e-6):
    n, p  = X.shape
    theta = np.zeros(1 + p)
    theta[0] = 100.0      # initial scale guess

    for k in range(max_iter):
        g     = gradient(X, t, delta, theta)
        H     = hessian(X, t, delta, theta)
        try:
            # H^{-1} g      never form H^{-1} explicitly
            step = np.linalg.solve(H, g)
        except np.linalg.LinAlgError:
            print("Singular Hessian: check for dummy-variable trap.")
            break
        theta -= step
        if np.linalg.norm(step) < tol:
            print(f"Converged in {k+1} iterations.")
            break

    return theta
```

Note: `np.linalg.solve(H, g)` is preferable to `np.linalg.inv(H) @ g`; it is numerically more stable and avoids explicitly forming H^{-1}.

9 Convergence: Why NR Wins in Under Ten Iterations

Running both optimisers on a trimmed maintenance dataset confirms the theory. Newton-Raphson declares convergence ($\|\text{step}\|_2 < 10^{-6}$) in under ten iterations. Gradient Descent, with a carefully chosen learning rate, requires several thousand iterations to reach the same tolerance. Both terminate at numerically identical $\hat{\boldsymbol{\theta}}$.

Why do they converge to the same point? The Weibull NLL is *strictly convex* over the feasible parameter space when X has full column rank. To see why: the second-order term ($\partial^2\mathcal{L}/\partial\alpha^2$) is a sum of positive terms near the optimum (where $u_i \approx \delta_i$ and $(\beta_i + 1)u_i > \delta_i$), and the γ-block of the Hessian is a positive semi-definite quadratic form in X that becomes definite under full rank. A strictly convex function has exactly one global minimum and no local traps. Both optimisers are simply taking different routes down the same bowl. What the Hessian buys Newton-Raphson is knowledge of the bowl's curvature at each step, enabling a nearly-optimal jump rather than GD's fixed, geometry-blind nudge.

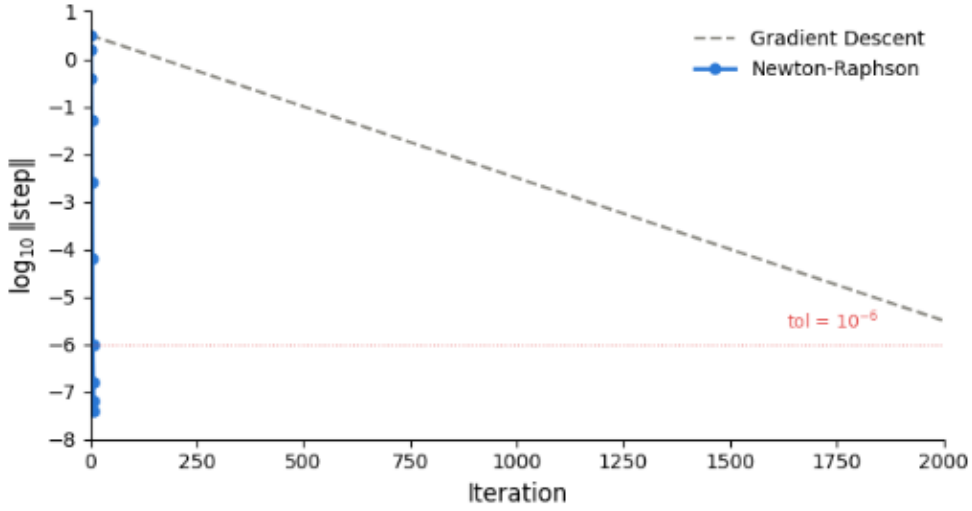


Figure 3: Convergence comparison. GD's fixed step size causes slow, geometry-blind descent; NR's curvature-corrected steps reach machine precision in fewer than ten iterations.

10 The Danger Zone: Intercept Columns and Hessian Singularity

When scaling from a single sensor to high-dimensional datasets with one-hot-encoded categorical variables, a structural statistical hazard appears.

Suppose we have a categorical variable with K levels and we one-hot-encode all K levels (creating K binary columns summing to 1 in every row), then also append a global column of 1s as an intercept γ_0. The intercept column is now a linear combination of the one-hot columns:

$$\mathbf{1} = \mathbf{c}_1 + \mathbf{c}_2 + \cdots + \mathbf{c}_K$$

This destroys the full column rank of X.

The numerical consequence is immediate: a rank-deficient X produces a rank-deficient Hessian, $\det(H) = 0$, so H cannot be inverted and the Newton-Raphson update is undefined. In practice, before reaching true singularity, the Hessian entries collapse toward zero and `np.linalg.solve` returns a solution with astronomically scaled parameter variances—Standard Errors beyond 1.3×10^6 and p-values washing out to an uninformative 0.999.

The resolution follows directly from the rank argument: **drop the global intercept column**. Our Weibull model already carries an intrinsic scale anchor in α, so γ_0 is genuinely redundant—not just numerically problematic. Removing it restores full column rank to X, positive definiteness to H, and yields clean, statistically significant parameter estimates.

This is also why `manual_weibull_optimization` above takes raw sensor features in X with no appended 1s: the intercept column is not "optional convention" but a structural correctness requirement.

11 Conclusion

The Weibull likelihood surface is not an abstraction—it is a concrete geometric object, and understanding its curvature is what separates a practitioner who tunes hyperparameters by intuition from

With intercept [rank-deficient]

1	cat_A	cat_B	cat_C	torque	rpm
1	1	0	0	42.1	1450
1	0	1	0	38.7	1380
1	0	0	1	51.3	1510
1	1	0	0	45.9	1490
1	0	1	0	33.2	1420

Intercept dropped [full rank ✓]

cat_A	cat_B	cat_C	torque	rpm
1	0	0	42.1	1450
0	1	0	38.7	1380
0	0	1	51.3	1510
1	0	0	45.9	1490
0	1	0	33.2	1420

Figure 4: The dummy-variable trap. Including both a global intercept and complete one-hot dummies creates a linear dependency in X, collapsing the Hessian rank. Dropping the intercept column restores full rank.

one who can read the Hessian and immediately diagnose whether the optimiser is stalling because of a flat plateau, a singular matrix, or genuine multicollinearity in the data.

By deriving every gradient and Hessian entry from scratch, we exposed three things that black-box libraries conceal:

1. Newton-Raphson converges in a handful of iterations because $H^{-1}\nabla\mathcal{L}$ computes the exact step that a second-order Taylor expansion says will reach the minimum in one move, and the Weibull NLL is smooth enough that this approximation is nearly exact near the optimum.
2. The dummy-variable singularity is not a numerical accident—it is a direct consequence of the Hessian losing rank when X loses linear independence. The fix is derivable from first principles, not guessable from trial and error.
3. The MLE exists and is unique because the NLL forms a strictly convex bowl over the feasible parameter space—a property both optimisers exploit but neither surfaces to the user.

This is the practical value of first-principles derivation in survival analysis: when the model breaks, you know exactly which equation is responsible.